Visual Book QA

Project Technical Report

Transforming Dense Documentation into Publication-Quality Visual Insights

Document Type	Technical Project Report
Version	Prototype
Track	Generative Ai Professional
Subject	AI-Powered Visual PDF Question Answering System

Table of Contents

1. Executive Summary

Visual Book QA is an end-to-end AI-powered application that fundamentally changes how users interact with PDF documents. Rather than reading through pages manually or receiving raw text snippets from a search engine, users upload any PDF, ask a natural language question, and receive a beautifully rendered visual slide as the answer.

The system addresses a persistent problem in knowledge work: dense technical books, research papers, and documentation are time-consuming to navigate, and existing query tools return unstructured text that still requires significant cognitive effort to interpret. Visual Book QA eliminates this friction by combining state-of-the-art retrieval techniques with a local language model and a deterministic rendering pipeline, producing structured, publication-quality slides on demand.

The project is built entirely on open-source components, runs fully locally with zero API keys, and is designed with guardrails that prevent hallucinations from reaching the rendered output.

2. Problem Statement

Professionals across research, education, and industry regularly need to extract specific knowledge from lengthy PDF documents. The current landscape of tools presents two inadequate extremes:

- Manual reading — thorough but slow. Finding a specific concept in a 500-page textbook can take hours.
- Keyword search / RAG tools — fast but visually barren. Tools return raw text snippets that require the user to mentally synthesise the content into a coherent understanding.

Neither approach closes the gap between data retrieval and genuine comprehension. Visual Book QA was designed to fill that gap by returning not just the right information, but the right information in a format that is immediately understandable — a structured slide with titles, key points, and diagrams that match the nature of the question.

3. System Overview

Visual Book QA is a four-stage pipeline that transforms a raw PDF and a natural language question into a rendered visual slide. Each stage is independently implemented as a Python module, enabling clean separation of concerns, testability, and future extensibility.

3.1 High-Level Architecture

The pipeline consists of four sequential modules:

- Ingestion — PDF parsing, chunking, embedding, and dual-index construction.
- Retrieval — Hybrid BM25 + FAISS search, RRF fusion, and cross-encoder reranking.
- Generation — LLM-driven HTML/SVG slide synthesis with a relevance gate.
- Rendering — Headless Chromium screenshot capture and PNG validation.

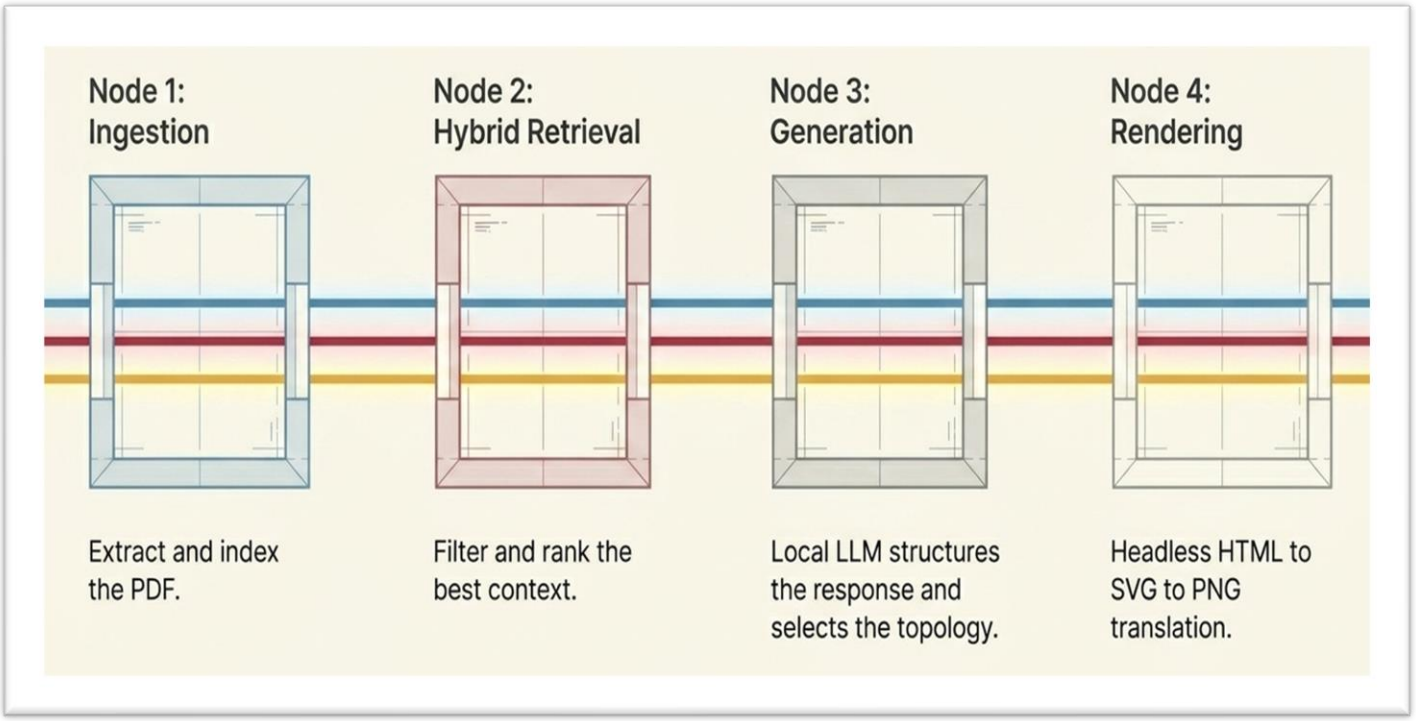

Figure 1. Project Architecture

A Gradio web interface ties the four modules together, exposing a simple two-panel UI: PDF upload and question input on the left, visual slide output on the right. All processing happens locally no data ever leaves the user's machine.

4. Pipeline Modules

4.1 Module 1 — Intelligent Ingestion

The ingestion module (ingestion.py) is responsible for converting a raw PDF file into a pair of search indexes that support both semantic and keyword retrieval. It is designed to be fast on repeat runs through a disk-based caching strategy.

4.1.1 Text Extraction

The module uses PyMuPDF (fitz) to open the PDF and extract plain text from every page. Pages are joined with newline separators to preserve document structure before chunking.

4.1.2 Chunking Strategy

Text is split into overlapping character-level chunks using a sliding window:

- Chunk size: 400 characters
- Overlap: 80 characters between consecutive chunks
- Minimum length filter: chunks shorter than 30 characters are discarded

The overlap ensures that concepts split at a chunk boundary are still represented in at least one complete chunk, reducing the risk of partial context reaching the retrieval stage.

4.1.3 Embedding and Indexing

Each chunk is embedded using the all-MiniLM-L6-v2 sentence-transformer model, producing 384-dimensional, L2-normalised vectors. These are stored in a FAISS IndexFlatIP index, which computes inner-product similarity (equivalent to cosine similarity on normalized vectors).

In parallel, a BM25Okapi index is built over the tokenized chunks using the rank-bm25 library. This sparse index captures exact keyword matches that dense embeddings may miss, especially for technical terms, acronyms, and proper nouns.

4.1.4 Caching

All artifacts the chunks list, the FAISS index, and the BM25 model are cached to disk under cache/<md5_hash>/. On subsequent runs with the same PDF, the module detects the cache hit and skips all processing, making repeat queries near-instant.

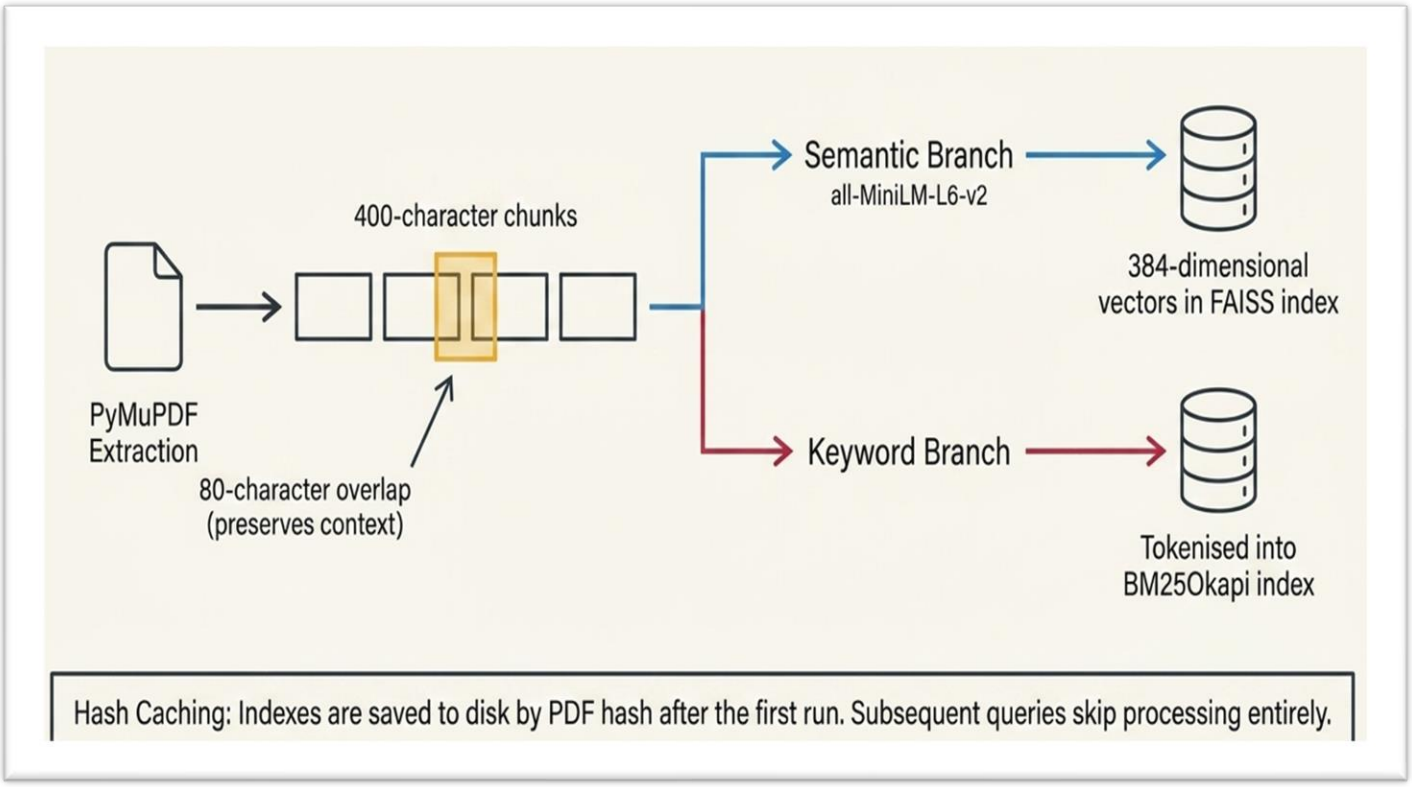

Figure 2. Ingestion Process

4.2 Module 2 — Three-Stage Retrieval Funnel

The retrieval module (retrieval.py) implements a hybrid retrieval pipeline that combines the complementary strengths of dense vector search and sparse keyword search, then applies a neural reranked for final precision.

4.2.1 Dense Retrieval (FAISS)

The user's question is encoded by the same all-MiniLM-L6-v2 model used during ingestion. The resulting query vector is searched against the FAISS index using inner-product similarity, returning the top 10 candidate chunks.

4.2.2 Sparse Retrieval (BM25)

The query is tokenized and scored against the BM25 index. The top 10 candidates with a BM25 score are retrieved independently of the dense results. This step is critical for queries that contain rare or domain-specific terms that the embedding model may not handle well.

4.2.3 Reciprocal Rank Fusion (RRF)

The two ranked lists are merged using Reciprocal Rank Fusion (RRF) with a rank constant k=60. RRF assigns each candidate a score of 1 / (k + rank) from each list and sums the scores. This method is robust to score scale differences between the two retrieval systems and consistently outperforms score-based fusion in practice.

4.2.4 Cross-Encoder Reranking

The fused candidate set is reranked by a cross-encoder model (ms-marco-MiniLM-L-6-v2). Unlike the bi-encoder used for embedding, the cross-encoder jointly processes the query and each candidate chunk together, enabling it to model fine-grained relevance relationships. The top 4 chunks by cross-encoder score are returned to the generation stage.

4.3 Module 3 — Guard Railed Generation

The generation module receives the top 4 retrieved chunks and the user question and uses a local LLM (Qwen 2.5-7B served via Ollama) to synthesize an HTML/SVG slide.

4.3.1 Relevance Gate

Before calling the LLM, the module evaluates whether the retrieved context is relevant to the question. If the relevance check fails, the pipeline halts and reports that no relevant content was found rather than fabricating an answer. This gate is the primary anti-hallucination control in the system.

4.3.2 Adaptive Slide Layout

The LLM is prompted to produce structured HTML with embedded SVG where appropriate. The layout adapts to the nature of the question: a conceptual question produces a title-and-bullets card, while a process question produces a flow diagram, and a comparison question produces a side-by-side table. The prompt explicitly instructs the model not to write graphical code SVG diagrams are assembled by a deterministic builder from a JSON specification produced by the LLM.

4.3.3 Deterministic SVG Building

A separate deterministic SVG builder (renderer.py) assembles diagram elements flowcharts, timelines, networks, cycle diagrams from the LLM's JSON output. By separating semantic content generation from graphical rendering, the system eliminates an entire class of LLM graphical hallucinations. The LLM only decides what to show; the code decides how to draw it.

4.4 Module 4 — Headless Rendering

The rendering module (renderer.py) converts the generated HTML string into a pixel-perfect PNG image using Playwright with a headless Chromium browser.

4.4.1 Rendering Process

- The HTML is written on a temporary file.
- A Playwright browser instance opens the file at a fixed 960x640 viewport.
- The page waits for the network idle state to ensure all resources are loaded.
- A full screenshot is taken and saved to outputs/.
- Pillow validates the PNG file integrity before returning the path.

The rendered HTML is also saved as outputs/last_debug.html, enabling developers to inspect and debug the slide source without running the full pipeline.

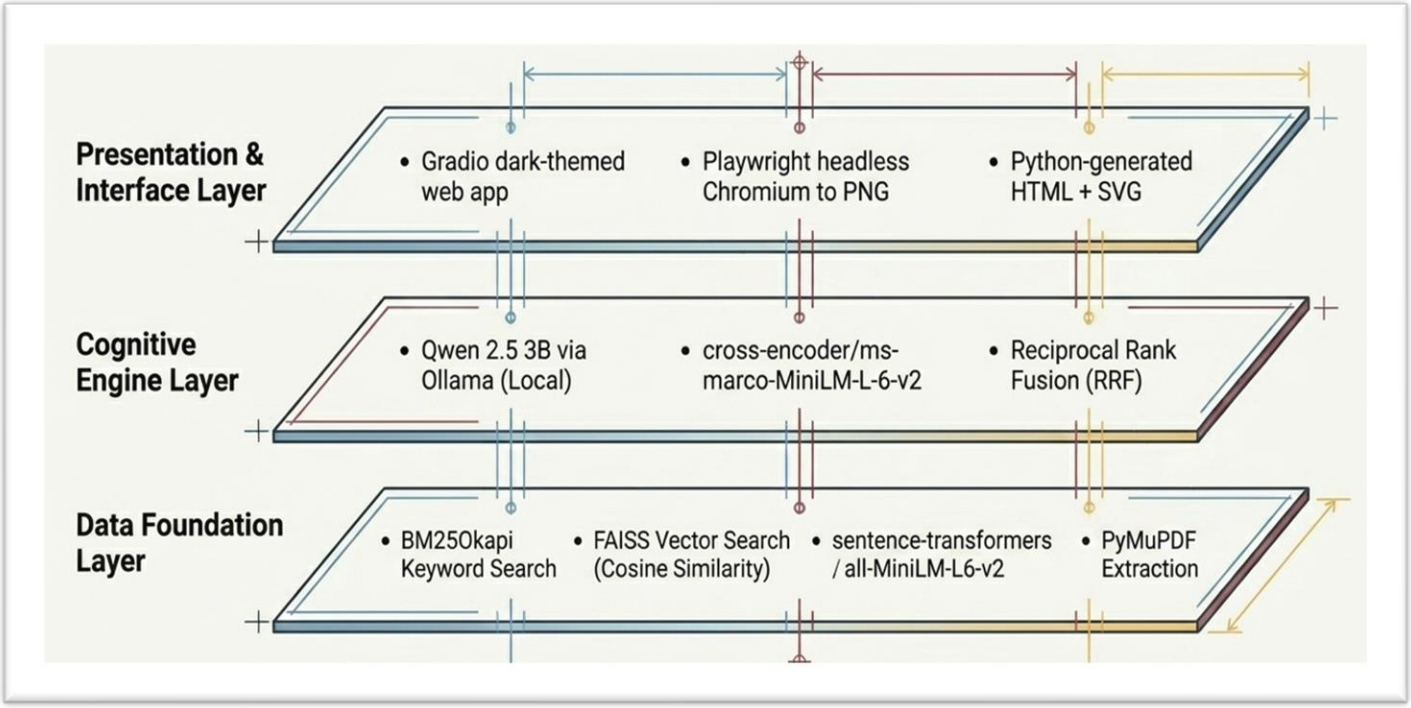

Figure 3. Rendering Layers

5. Comparison with Traditional Approaches

The following table summarizes how Visual Book QA compares to conventional PDF question-answering tools across four key dimensions:

Feature	Traditional PDF QA	Visual Book QA
Output Format	Raw text snippets	**Structured visual slide + diagram**
Data Privacy	Cloud / API dependent	**Fully local — zero API keys**
Generative Accuracy	Prone to hallucination	**Guardrailed & deterministic**
Time to Value	Manual synthesis required	**Instant generation**

6. Technology Stack

Visual Book QA is built entirely on open-source components. No proprietary APIs, cloud services, or external data transmission are required. The full dependency stack is as follows:

Component	Technology	Purpose
PDF Parsing	**PyMuPDF (fitz)**	Page-by-page text extraction
Vector Index	**FAISS (IndexFlatIP)**	Cosine-similarity semantic search
Keyword Index	**BM25Okapi (rank-bm25)**	Sparse keyword retrieval
Embedding Model	**all-MiniLM-L6-v2**	384-dim sentence embeddings
Reranker Model	**ms-marco-MiniLM-L-6-v2**	Cross-encoder precision scoring
LLM Generation	**Qwen 2.5-7B via Ollama**	HTML/SVG slide generation
HTML Rendering	**Playwright + headless Chromium**	PNG screenshot from HTML
Image Validation	**Pillow**	PNG integrity verification
UI Framework	**Gradio**	Web interface
Caching	**FAISS + pickle on disk**	Re-run acceleration per PDF

7. User Interface

The user interface is built with Gradio and features a dark-themed two-panel layout designed for clarity and minimal cognitive load.

7.1 Left Panel — Controls

- PDF Upload: A drag-and-drop file picker that accepts PDF files. On upload, the file is immediately ingested, and the chunk count is displayed in a status badge.
- Question Input: A multiline text area with placeholder examples. Questions can be submitted by pressing Enter or clicking the Generate Slide button.
- Generate Slide: Primary action button that triggers the full retrieval-generation-rendering pipeline.
- Regenerate: Secondary button that reruns generation with the same question, producing an alternative slide layout.
- Status Bar: Inline feedback showing pipeline progress and error messages.

7.2 Right Panel — Output

The right panel occupies two-thirds of the layout and displays the rendered PNG slide. The image is automatically scaled to fill the panel and is accompanied by a download affordance. Users can inspect the raw HTML source in the outputs/last_debug.html file for debugging.

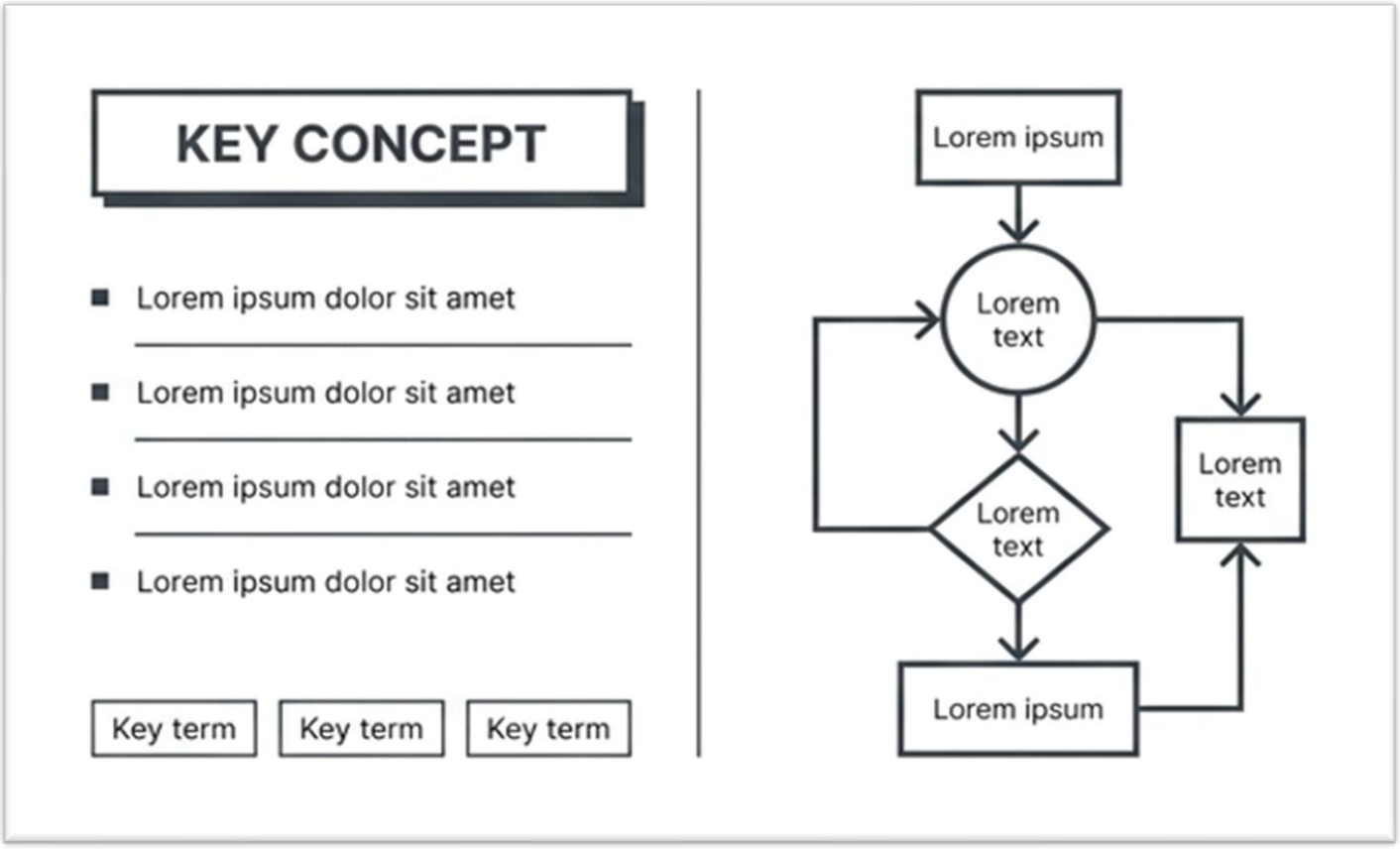

Figure 4. Slide Output

8. Key Design Decisions

8.1 Fully Local Execution

Every component in the pipeline runs locally. The embedding model, the reranked, and the LLM are all downloaded once and cached on disk. This design ensures absolute data privacy, no document content, text question, or generated answer ever leaves the user's machine. It also eliminates API costs and latency variability.

8.2 Hybrid Retrieval over Pure Dense Search

Pure vector search can fail for queries containing rare technical terms or precise identifiers that may not be well-represented in the embedding space. BM25 complements dense retrieval in exactly these cases. The RRF fusion step avoids the need to tune score thresholds across the two systems, producing a stable merged ranking.

8.3 Cross-Encoder Reranking

Bi-encoder models (used for FAISS retrieval) encode query and document independently, making them fast but less precise than cross-encoders that model query-document interaction jointly. Using the cross-encoder only at the reranking stage over a small candidate set achieves the precision of cross-encoders at a fraction of the latency cost.

8.4 Deterministic SVG vs. LLM-Written Graphics

Asking an LLM to write SVG or HTML directly leads to malformed markup, incorrect coordinates, and layout errors. Visual Book QA separates concerns: the LLM produces a semantic JSON description of the diagram, and a deterministic Python builder translates that description into pixel-perfect SVG. This eliminates graphical hallucinations entirely.

8.5 Disk-Based Caching

Re-running the embedding and indexing pipeline for a large PDF can take tens of seconds. The ingestion module caches all artifacts keyed by an MD5 hash of the PDF content. Repeat queries against the same document skip ingestion entirely, making the interactive experience fast from the second query onwards.

9. Conclusion

Visual Book QA demonstrates that the gap between document retrieval and visual comprehension can be closed with a carefully engineered, fully local pipeline. By combining hybrid retrieval, neural reranking, guard railed generation, and deterministic rendering, the system produces structured visual answers that are ready to share, present, or embed without any manual post-processing.

The architecture is intentionally modular: each of the four pipeline stages can be upgraded independently. The retrieval funnel can be extended with metadata filtering or chunk summarization; the generation stage can be adapted to different LLMs or output formats; the rendering stage can be extended to produce PDF slides or PowerPoint exports.

Visual Book QA represents a new standard for document intelligence, one where the answer to a question is not a wall of text, but a publication-quality visual that communicates immediately.

10. Future Improvement

10.1 Advanced Retrieval & Context Management

The current retrieval funnel uses a three-stage process (BM25, FAISS, and Cross-Encoder). Future iterations could improve accuracy by:

- Metadata Filtering: Integrating document metadata (e.g., chapter titles, page numbers) into the retrieval process to provide better contextual grounding.
- Chunk Summarization: Implementing a "summary-aware" retrieval where the system generates brief summaries of chapters to help the model decide which sections are most relevant before diving into specific 400-character chunks.
- Recursive Retrieval: Allowing the system to "reason" by performing multi-hop retrieval if a complex question requires information from two different parts of a book.

10.2 Enhanced Visual & Diagrammatic Capabilities

The "Deterministic SVG Building" module is a key design decision that prevents hallucinations. This can be scaled by:

- Expanded Diagram Library: Adding more complex deterministic templates, such as Gantt charts, Venn diagrams, or 3D architectural representations, based on the LLM's JSON output.
- Multi-Slide Generation: Developing the ability to synthesize an entire "summary deck" (multiple sequential slides) rather than a single response slide for broad questions.

10.3 Performance & Scalability

The system currently focuses on fully local execution to ensure privacy. Future technical refinements include:

- Quantization & Model Optimization: Testing smaller or more highly quantized versions of models (like Qwen 2.5) to enable the pipeline to run on lower-end consumer hardware or mobile devices.

- Asynchronous Processing: Implementing a task queue to allow the "Intelligent Ingestion" of multiple large PDFs in the background while the user interacts with previously cached documents.

10.4 User Experience (UX) Enhancements

To move beyond the current two-panel Gradio interface:

- Source Citations: Adding a visual link between the generated slide and the specific page in the uploaded PDF, perhaps using a split-screen view that highlights the source text.

- Conversational Memory: Enabling the system to remember previous questions and slides so users can "iterate" on a design (e.g., "Change the flowchart to a table" or "Add more detail to the third bullet point").

— End of Report —